# PYTHONPROGRAMMING

## MODULE3

## Dictionaries,NumPy,Files

# MODULE3

## Module3:Dictionaries,Numpy,Files

**Syllabus:**

- **Dictionaries:**Dictionaryoperations,dictionarymethods,aliasingandcopying .
- **Numpy:**About,Shape,Slicing,masking,Broadcasting,dtype.
- **Files:**Aboutfiles,writingourfirstfile,readingafileline-by-line,turningafileintoalistoflines,readingthewholefileatonce,workingwithbinaryfiles,Directories,fetchingsomethingfromtheWeb.
- **Chapter:**5.4,6.1-6.5,7.1-7.8
- **TextBook:**PeterWentworth,JeffreyElkner,AllenB.DowneyandChrisMeyers-Howtothinklikeacomputerscientist:learningwithPython3. GreenTeaPress,Wellesley,Massachusetts,2020

---

## Chapter5.4:Dictionaries

**Introduction:**

- Dictionariesareabuilt-indatatype,usedtostoredatavaluesinkey:valuepairs.
- Eachkey:valuepairiscalledanelementofthedictionary.
- Dictionariesarewrittenwithcurlybracketsandhavekeysandvalues.
- Thekey:valuepairsofthedictionaryareseparatedbycommas.Each paircontainsakeyandavalueseparatedbyacolon.
- Indexesfordictionariesarecalledkeys.
- Dictionaryitemsareordered,changeable,anddonotallowduplicates.
- Thekeys,whichcanbeanyimmutabletype,andthevaluesaremutable.
- Dictionariescanusemanydifferentdatatypes,notjustintegers.
- Examples:

```
fruits={"apples":430,"bananas":312,"oranges":525,"pears":217}
```


country_capitals = {
'Germany' : 'Berlin',
element 1
'Canada' : 'Ottawa',
element 2
'England' : 'London'
element 3
}
key
value

**Dictionaryoperations**

- Dictionarieshaveseveralusefulbuilt-inoperations.Commonoperationsinclude
  1. CreatingaDictionary
  2. AccessingDictionaryitems
  3. ModifyingDictionaryitems
  4. DeletingDictionaryitems.

**1. CreatingaDictionary:**

InPython,dictionariesstoredataaskey-valuepairs.Itiscreatedusingcurlybraces({}).Examplecapitals={"Maharashtra":"Mumbai",Telangana":"Hyderabad","Karnataka":"Bengaluru"}.

**2. AccessingDictionary:**

- AccessingValuesbyKey:Themostdirectwaystoretrieveavalueassociatedwithaspecifickeyisusingsquarebrackets[].Example

```
>>>my_dict={'fruit':'apple','colour':'blue','age':25}
>>>print(my_dict['fruit'])#Output:apple
```

**3. ModifyingDictionaryitems:**

- InPython,dictionariesaremutable,andyoucanmodifythembychangingexistingvalues,addingnewkey-valuepairs,orremovingitems.Wecanchangethevalueofaspecificitembyreferringtoitskeyname.Forexample,changethe"year"to2018inthefollowingdictionary.

```
>>>thisdict={"brand":"Ford","model":"Mustang","year":1964}
>>>thisdict["year"]=2018
>>>thisdict={"brand":"Ford","model":"Mustang","year":2018}
```

- Theupdate()methodwillupdatethedictionarywiththeitemsfromthegivenargument.Forexample,updatethe"year"ofthecarbyusingtheupdate()method:

```
>>>  thisdict={"brand":"Ford","model":"Mustang","year":1964}
>>>  thisdict.update({"year":2020})
>>>  thisdict={"brand":"Ford",model":"Mustang","year":2020}
```

- updatethedictionarywiththeitems.Example

```
>>>inventory={"apples":430,"bananas":312,"oranges":525,
                    "pears":217}
>>>inventory["bananas"]=50
>>>print(inventory)#Output
```

Dr. Suresha V

```
{'pears':217,'apples':430,'oranges':525,'bananas':50}
>>>inventory["bananas"]+=200
>>>print(inventory)
{'pears':0,'apples':430,'oranges':525,'bananas':250}
```

- The `len()` function also works on dictionaries; it returns the number of key:value pairs. Example

```
>>>inventory={"apples":430,"bananas":312,"oranges":525,
"pears":217}
>>>len(inventory)#Output4
```

4. **Deleting Dictionary items:** We can delete items from a dictionary using **del** dictionary[key] for a specific key. Example

```
>>>inventory={"apples":430,"bananas":312,"oranges":525,
"pears":217}
>>>delinventory["bananas"]
>>>print(inventory)    #Output
{'apples':430,'oranges':525,'pears':217}
```

---

**Dictionary methods:** Dictionaries have several useful built-in methods.

1. **keys()** method
2. **values()** method
3. **items()** method

- Accessing All Keys, Values, or Items: We can access all components of a dictionary using methods.
- **keys()** method: It returns a view of all keys in the dictionary, which are often used in loops. example

```
>>>my_dict={'fruit':'apple','colour':'blue'}
>>>forkeyinmy_dict.keys():
>>>print(key)#Output:fruit,color
```

- **values()** method: it returns a view of all values in the dictionary.

```
>>>my_dict={'fruit':'apple','color':'blue'}
>>>forkeyinmy_dict.values():
>>>print(key)#Output:apple,blue
```

- **items()** method: Returns a view of all key-value pairs as a list of tuples. This is ideal for iterating through both keys and values simultaneously.

Dr. Suresha V

Example:

```
>>>my_dict={'fruit':'apple','color':'blue'}
>>>forkey,valueinmy_dict.items():
>>>print(f"Thekeyis{key}andthevalueis{value}")#Output:
 #ThekeyisfruitandthevalueisapplethekeyIs
 colorandthevalueisblue
```

- **Theinandnotinoperatorsinthedictionary**:Theseoperatorsareusedtotestifakeyisinthedictionary.Example

```
>>> my_dict = {'apple': 1, 'banana': 2, 'cherry': 3}
>>> 'apple' in my_dict
True
>>> 'orange' in my_dict
False
>>> 'orange' not in my_dict
True
```

### 5.4.3Aliasingandcopying

- **Aliasing**
  - Aliasingoccurswhenmultiplevariablesrefertothesamedictionaryobjectinmemory.
  - Sincedictionariesaremutable,changesmadethroughonevariablewillbereflectedinallothervariablesthatpointtothatsameobject.
  - Thismeansthatchangesmadetotheobjectthroughonevariablenamewillbereflectedinallothervariablesthatreferencethesameobject.Example

```
>>>original_dict={'a':1,'b':2}
>>>alias_dict=original_dict      #Aliasingoccurshere
>>>alias_dict['b']=99      #Changemadethroughthealias
>>>print(original_dict['b'])#Output:99-theoriginalisaffected
```

- **copying**
  - InPython,thecopy()methodisabuilt-indictionarymethodusedtocreatea**shallowcopy**ofadictionary.
  - Ashallowcopycreatesanewdictionaryobject,sochangestothetop-levelkeysinthecopydonotaffecttheoriginal,andviceversa.
  - Ifwewanttomodifyadictionaryandkeepacopyoftheoriginal,usethecopymethod.

- Example:

```
>>>opposites={"up":"down","right":"wrong","yes":"no"}
>>>alias=opposites
>>>copy=opposites.copy()#Shallowcopy
```

- aliasandoppositesrefertothesameobject;copyreferstoafreshcopyofthesame dictionary.Ifwemodifythealias,oppositesisalsochanged

```
>>>alias["right"]="left"
>>>opposites["right"]'l
   eft'
```

- Ifwemodifyacopy,oppositesareunchanged.

```
>>>copy["right"]="privilege"
>>>opposites["right"]'l
   eft'
```

### 5.4.4Countingletters

- Therearevariousmethodstocountthefrequencyofcharactersinagivenstring. Onesimplemethodtocountthefrequencyofeachcharacterisbyusingadictionary.
- Theideaistotraversethrougheachcharacterinthestringandkeepacountofhow manytimesitappears.example
- WecancountthelettersinaPythonstringusingadictionarywithasimpleforloopa ndtheget()method.Example

```
s = "Mississippi"
freq = {}
for c in s:
    if c in freq:
        freq[c] += 1
    else:
        freq[c] = 1
print(freq)
```

Output:

```
{'M': 1, 'i': 4, 's': 4, 'p': 2}
```

- Itmightbemoreappealingtodisplaythefrequencytableinalphabeticalorder.W ecandothatwiththeitemsandsortmethods.

```
>>>letter_items=list(letter_counts.items())
>>>letter_items.sort()
>>>print(letter_items)
[('M',1),('i',4),('p',2),('s',4)]
```

- **DifferenceBetweenMutableandImmutableDataType**

| | Mutable | Immutable |
|---|---|---|
| **Definition** | Data type whose values can be changed after creation. | Data types whose values can't be changed or altered. |
| **Memory Location** | Retains the same memory location even after the content is modified. | Any modification results in a new object and new memory location |
| **Example** | List, Dictionaries, Set | Strings, Types, Integer |
| **Performance** | It is memory-efficient, as no new objects are created for frequent changes. | It might be faster in some scenarios as there's no need to track changes. |
| **Thread-Safety** | Not inherently thread-safe. Concurrent modification can lead to unpredictable results. | They are inherently thread-safe due to their unchangeable nature. |
| **Use-cases** | When you need to modify, add, or remove existing data frequently. | When you want to ensure data remains consistent and unaltered. |

- **CompareListversusDictionariesDataTypes**

| Lists | Dictionaries |
|---|---|
| List is an ordered collection of items. | Dictionaries are an unordered collection of data in a key: value pair form. |
| Uses square brackets [ ]. | Uses curly braces { }. |
| Ex: fruits = ["apple", "banana", "cherry"] | Ex: person = {"name": "John", "age": 30} |
| Ordered: Items have a defined order, which will not change. | Unordered: Items do not have a defined order. |
| Accessed by index, starting from 0. | Values are accessed using keys. |
| Allows duplicate items. | Does not allow duplicate keys. However, values can be duplicated. |
| It can store any data type. | Keys can be of any immutable data type (e.g., strings, numbers, tuples). Values can be of any type. |
| Built in functions: append(), remove(), pop(), sort(), etc. | Built in functions: keys(), values(), items(), get(), pop(), etc. |
| Slicing can be done | Dictionaries can not be sliced. |

## Chapter6:NumPy

**Introduction:**

- NumPystandsforNumericalPython.
- NumPyisaPythonlibraryusedforworkingwitharrays.
- Itprovidessupportforlarge,multi-dimensionalarraysandmatrices,alongwithavastcollectionofhigh-levelmathematicalfunctionstooperateonthesearraysefficiently.
- ItisafundamentallibraryinPythonforscientificcomputing.

- **WhyNumPy**

- Itprovidesahigh-performance,multi-dimensionalarrayobjectandacomprehensivelibraryoffunctionsdesignedfornumericalcomputing.
- Itissignificantlyfasterandmorememory-efficientthanstandardPythonlists,makingitthefundamentalpackageforscientificcomputing.
- Itoffersavastcollectionofhigh-levelmathematicalfunctionsfortaskssuchas:
  - Linearalgebra(e.g.,matrixmultiplication,inversion,eigenvalues).
  - Statisticaloperations(e.g.,mean,median,standarddeviation).
  - Fouriertransformsandrandomnumbergeneration.
- MemoryEfficiency:NumPyarraysuselessmemorythanPythonlistsforstoringlarge,homogeneousnumericaldata,whichiscrucialwhenworkingwithlargedatasets.
- ThestandardPythondatatypesarenotverysuitedformathematicaloperations. Example:Forexample,supposewehavethelista=[2,3,8].Ifwemultiplythislistbyaninteger,weget:

```
>>>a=[2,3,8]
>>>2*a
   [2,3,8,2,3,8]
```

- Supposemultiplyingbyafloatisnotevenallowed:

```
>>>a=[2,3,8]
>>>2*a
>>>2.1*a
TypeError:can'tmultiplysequencebynon-intoftype'float'
```

- ThisisbecausePythonlistsarenotdesignedasmathematicalobjects.Rather,theyarepurelyacollectionofitems.Inordertogetatypeoflistthatbehaveslikeamathematicalarrayormatrix,weuseNumpy.

- NumPyaimstoprovideanarrayobjectthatisupto50xfasterthantraditionalPythonlistsbecauseNumPyarraysarestoredatonecontinuousplaceinmemory,unlikelists,soprocessescanaccessandmanipulatethemveryefficiently.
- Arraysareveryfrequentlyusedindatascience,wherespeedandresourcesareveryimportant.

---

- **NumPyGettingStarted:**

- OnceNumPyisinstalled,itcanbeusedbyaddingtheimportkeyword:importnumpy

  **>>>importnumpy#**NowNumPyisimportedandreadytouse.

- Example

```
>>>importnumpy
>>>arr=numpy.array([1,2,3,4,5])
>>>print(arr)#    output[12345]
```

- NumPyasnp:NumPyisusuallyimportedunderthenpalias.Analiasmeansanalternatenameforreferringtothesamething.Createanaliaswiththeaskeywordwhileimporting:

```
>>>importnumpyasnp
```

- NowtheNumPypackagecanbereferredtoasnpinsteadofnumpy.

```
>>>importnumpyasnp
>>>arr=np.array([1,2,3,4,5])
>>>print(arr)#    output[12345]
```

- Togetatypeoflistthatbehaveslikeamathematicalarrayormatrix,weuseNumpy.Example1

```
>>>importnumpyasnp
>>>a=np.array([2,3,8])
>>>2.1*a
array([4.2,6.3,16.8])
```

- Thelist[2,3,8]inaboveexamplecontainsint's,yettheresultcontains floats.Thismeansnumpychangedthedatatypeautomaticallyforus.

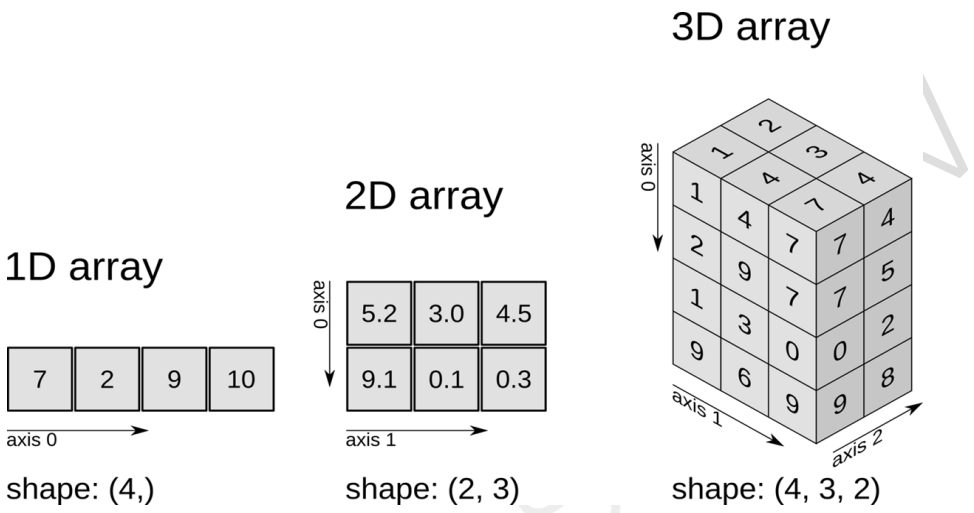
- Example2:multiplytogetherarrays.

```
>>>importnumpyasnp
>>>a=np.array([2,3,8])
>>>a*a
    array([4,9,64])
>>>a**2
    array([4,9,64])
```

Dr. Suresha V

**Shape**

- Oneofthemostimportantpropertiesanarrayisitsshape.
- InNumPy,theshapeofanarrayreferstothenumberofelementsalongeachdimensionofthearray.Itisrepresentedasatuple,whereeachintegerinthetupleindicatesthesizeofthecorrespondingdimension.
- Example:




- Togettheshapeofanarray,weuseshape:
  - **Example1:**

```
>>>importnumpyasnp
>>>a=np.array([2,3,8])
>>>a.shape#output(3,)
```

  - **Example2:**

```
>>>b=np.array([[2,3,8],[4,5,6],])
>>>b.shape#output(2,3)
```

  - **Example3:**

```
importnumpyasnpy
#creatinga2-darray
arr1=npy.array([[1,3,5,7],[2,4,6,8]])
#creatinga3-darray
arr2=npy.array([[[1,2],[3,4]],[[5,6],[7,8]]])
print(arr1.shape)
print(arr2.shape)
#Output:
 (2,4)
 (2,2,2)
```

Dr.Sure

## Slicing

- NumPy array slicing isa powerful method for extracting specificportions(subsets)ofdatafromanarray.
- Thegeneralsyntaxforslicingis**arr[start:stop:step]**
- Thistechniqueappliestoarraysofanydimension**.**
- For1Darrays,itworksjustlikefornormalPythonlists.Example

```
>>>a=np.array([2,3,8])
>>>a[2]#Output8
>>>a[1:]#np.array([3,8])
```

- However,whendealingwithhigher-dimensionalarrays,somethingelsehappens:

```
>>>b=np.array([[2,3,8],[4,5,6],])
>>>b[1]#Outputarray([4,5,6])
>>>b[1][2]#Outputarray6
```

- Byusingb[1],returnsthe1strowalongthefirstdimension,whichisstillanarray.Afterthat,wecanselectindividualitemsfromthat.Thiscanbeabbreviatedto:

```
>>>b[1,2]#Output6
```

- ButwhatifIwantedthe1stcolumninsteadofthefirstrow?Thenweuse:toselect allitemsalongthefirstdimension,andthena1:

```
>>>b[:,1]#array([3,5])
```

## Masking

- MaskinginNumPyreferstotheprocessofselectivelyworkingwithelementswithinanarraybasedonabooleancondition.
- ThisisdoneusingaBooleanmask(anarrayofTrue/Falsevalues)
- Thistechniqueisusedtofilter,select,ormodifyelementsofanarraybasedonaBooleancondition.
- Supposewehaveanarray,andwewanttothrowawayallvaluesaboveacertaincutoff.**Forexample**

```
>>>a=np.array([230,10,284,39,76])
>>>cutoff=200
>>>a>cutoff
    np.array([True,False,True,False,False])
```

- **Example2:**FilteringData:Extractingelementsthatmeetacertaincondition.

Dr. Suresha V

```
importnumpyasnp
arr=np.array([10,20,30,40,50])mask=arr
>25
filtered_arr=arr[mask]#SelectselementswheremaskisTrue#Output:
[304050]
```

- **Example3:**ModifyingElements:Changingvaluesinanarraybasedonacondition.

```
importnumpyasnp
arr=np.array([10,20,30,40,50])
arr[arr>30]=100#Replacevaluesgreaterthan30with100
#Output:[10     20  30100100]
```

- **Example4**:MultipleConditions:Usingbitwiselogicaloperators(&forAND,|forOR,~forNOT)tocombineconditions.

```
arr=np.array([10,20,30,40,50])mask=(ar
r>15)&(arr<45)filtered_arr=arr[mask]
#Output:[203040]
```

**Broadcasting**

- NumPybroadcastingisamechanismthatallowsarithmeticoperationsonarraysofdifferentshapesandsizesbyautomaticallyexpandingthesmallerarraytomatchtheshapeofthelargerone
- Itenableselement-wiseoperationswithoutrequiringexplicitreshapingorrepeatingofdata,leadingtomoreconciseandefficientcode.
- Examples:

1.**BroadcastingaScalar:**Addingascalartoanarrayisthesimplestformofbroadcasting.Thescalarisconceptuallystretchedtomatchtheshapeofthearray.

```
importnumpyasnp
a=np.array([1,2,3])
b=2
#a.shapeis(3,),bisascalar
#NumPyconceptuallytreats'b'asanarrayofshape(3,)withvalues[2,2,2]
result=a+b      #resultis[3,4,5]
```

- Example2:Broadcastingtakesplacewhenyouperformoperationsbetween arraysofdifferentshapes.Forinstance

```
>>> a = np.array([
    [0, 1],
    [2, 3],
    [4, 5],
    ])
>>> b = np.array([10, 100])
>>> a * b
array([[  0, 100],
       [ 20, 300],
       [ 40, 500]])
```

- Intheaboveexample,theshapesofaandbdon'tmatch.Toproceed, Numpywillstretchbintoaseconddimension,asifitwerestackedthreetimesupo nitself.Theoperationthentakesplaceelement-wise.

## CHAPTER7:FILES

**Aboutfiles:**

- Variablesintheprogramsstoredatawhiletheprogramisrunning.
- Butifwewantourdatatopersistevenafterourprogramhasfinished,weneedtosa veittoafile.
- Afileisacollectionofdatastoredonadevicelikeaharddrive,whichcancontainan ythingfromdocumentsandimagestovideosandsoftwareinstructions.
- Pythonalsosupportsfilehandlingandallowsuserstohandlefiles,i.e.readandwri tefiles,alongwithmanyotherfilehandlingoptions.

- Dataonnon-volatilestoragemediaisstoredinnamedlocationscalledfiles.
- TherearemainlyTWOtypesofdatafiles:TextFilesandBinaryFiles
- TEXTFILE:Storesthedataintheformofastring.Atextfileconsistsofhuman-readablecharacters,whichcanbeopenedbyanytexteditor. Example.txt,.c,.cpp,.py
- BINARYFILE:Storesintheformofbytes.Binaryfilesaremadeupofnon-human-readablecharactersandsymbols,whichrequirespecificprogramstoaccessthei rcontents.Example.jpg,.gifor.png

- **Advantagesoffiles:**Dataisstoredpermanently,updatingbecomeseasy,Dat acanbesharedamongvariousprograms,AHugeamountofdatacanbestored.

Dr.Suresha V

- **Fileoperations:**InPython,afileoperationtakesplaceinthefollowingorder
  1. Openafile
  2. Readorwriteoperation
  3. Closethefile

## 7.2Writingourfirstfile

- InPython,filesarewrittenusingthebuilt-inopen()functionalongwithspecificfile modes('w'for write,'a' for append) andthewrite()orwritelines()methods.
- Itisbestpracticetousethewithstatementtoensurethefileisautomaticallyclosed.
- Twomodesforwriteoperation:
  1. Writemode'w':replacetheolddatawithnewdata
  2. Appendmode'a':appendthenewdatatoolddata
- Syntax:

  f=open('filename','w')

  f.write('Happy day!')

  f.close()
- Example:Writemode

| Program: | Output: |
|---|---|
| `import os`<br>`spam = open('hello25.txt','w')`<br>`print(spam.write('Good morning!'))`<br>`spam.close()` | `>>> %Run Tues25.py`<br>`13`<br>`Good morning!` |

- Anothersimpleprogramthatwritesthreelinesoftextintoafile:

**Program:** **Output:**

- Inthisexample,thevariablemyfilereferstothenewhandleobject.Online1,theopenfunctiontakestwoarguments.Thefirstisthenameofthefile,andthesecondisthemode.Mode"w"meansthatweareopeningthefileforwriting.Withmode"w",ifthereisnofilenamedtest.txtonthedisk,itwillbecreated.

- Iftherealreadyisone,itwillbereplacedbythefilewearewriting.Toputdatainthefileweinvokethewritemethodonthehandle,showninlines2,3and4above.

**7.3Readingafileline-at-a-time**

- Thefollowingmethodsareusedforreadoperations.
  1. **read():**readingthecontentofthefile.
  2. **read(integer):**ReadOnlyPartsoftheFile,canalsospecifyhowmanycharactersyouwanttoreturn
  3. **readline():**Readonelineofthefile,calling2timestoreadtwolines.
  4. **readlines():**readalllinesfromthefile.

**1. read():**readingthecontentofthefile.Thestepsinvolvedintheseoperationsare

- **Step 1:**Open your document file. For example, test.txt, whichcontainsthefollowingdatausing

```
Teacher: Why are you late for class, Rathod?

Rathod: Because of a sign board on the road.

Teacher: What does the sign have to do with you being late?

Rathod: The sign said, "School Ahead, Go Slow!"
```

- **Step2:**Toopentheabovefile,usethebuilt-inopen()function.
- **Step3:** Theopen()function returns a file object, which hasaread()methodforreadingthecontentofthefile,example

```
>>>importos
>>>f=open("test.txt",'r')
>>>print(f.read())
>>>f.close()
```

```
Teacher: Why are you late for class, Rathod?

Rathod: Because of a sign board on the road.

Teacher: What does the sign have to do with you being late?

Rathod: The sign said, "School Ahead, Go Slow!"
```

- Ifthesamefileislocatedinadifferentlocation,youwillhavetospecifythefilepath, likethis:

```
>>>importos
>>>f=open("C:\\Users\\myfiles\test.txt",'r')
>>>print(f.read())
>>>f.close()
```

- **readline():**Readonelineofthefile.Exampleusesthesametest.txtfile

```
>>>importos
>>>f=open("test.txt",'r')
>>>print(f.readline())
>>>f.close()
#Output:Itreadsthefirstlineofthedocument
```

```
Teacher: Why are you late for class, Rathod?
```

- **read(integer):**ReadOnlyPartsoftheFile,canalsospecifyhowmanycharacter syouwanttoreturn.example

```
>>>importos
>>>f=open("test.txt",'r')
>>>print(f.read(24))
>>>f.close()
#Output:Itreadsthefirst24charactersfromthedocument
```

```
Teacher: Why are you lat
```

### 7.4Turningafileintoalistoflines

- Itisoftenusefultofetchdatafromadiskfileandturnitintoalistoflines.
- Supposewehaveafilecontainingourfriendsandtheiremailaddresses,oneperline in the file. But we'd like the lines sorted into alphabetical order. A good planistoreadeverythingintoalistoflines,thensortthelist,andthenwritethesortedlistbackt oanotherfile:Example

```
import os

with open("friends.txt", "r") as input_file:
    all_lines = input_file.readlines()

all_lines.sort()

with open("sortedfriends.txt", "w") as output_file:
    for line in all_lines:
        outut_file.write(line)
```

- Thereadlinesmethodinline3readsallthelinesandreturnsalistofthestrings.

## 7.5Readingthewholefileatonce

- Anotherwayofworkingwithtextfilesistoreadthecompletecontentsofthefileintoastring,andthentouseourstring-processingskillstoworkwiththecontents.Considerthefollowingphrasesavedinafile,somefile.txt

```
Many useful line-processing programs will read a text file
line-at-a-time and do some minor processing as they write
the lines to an output file. They might number the lines in
the output file, or insert extra blank lines after every 60
lines to make it convenient for printing on sheets of paper,
or extract some specific columns only from each line in the
source file, or only print lines that contain a specific
substring. We call this kind of program a filter.
```

- Usethefollowingcodetoreadtheentirefileusingthereadfunctionandconvertitintoalistofwords,andcountthenumberofwordsinthedocuments.

```
with open("somefile.txt") as f:
    content = f.read()
    words = content.split()
    print("There are {0} words in the file.".format(len(words)))
```

- Theoutput

```
There are 82 words in the file.
```

## 7.6Anexample

- Manyusefulline-processingprogramswillreadatextfileline-by-lineanddosomeminorprocessingastheywritethelinestoanoutputfile.
- Theymightnumberthelinesintheoutputfile,orinsertextrablanklinesafterevery60linestomakeitconvenientforprintingonsheetsofpaper,orextractsomespecificcolumnsonlyfromeachlineinthesourcefile,oronlyprintlinesthatcontainaspecificsubstring.Wecallthiskindofprogramafilter.Hereisafilterthatcopiesonefiletoanother,omittinganylinesthatbeginwith#:

```
def filter(oldfile, newfile):
    with open(oldfile, "r") as infile, open(newfile, "w") as outfile:
        for line in infile:
            # Put any processing logic here
            if not line.startswith('#'):
                outfile.write(line)
```

- Online2,weopentwofiles:thefiletoread,andthefiletowrite.Fromline3,wereadtheinputfilelinebyline.Wewritethelineintheoutputfileonlyiftheconditiononline5istrue.

**Directories**

- ADirectory,sometimesknownasafolder,isaunitoforganisationalstructureina computer'sfilesystemforstoringandlocatingfilesorotherfolders
- Theyserveasawaytogroupandmanagefileshierarchically.
- Filesonnon-volatilestoragemediaareorganisedbyasetofrulesknownasafilesystem.
- Filesystemsaremadeupoffilesanddirectories,whicharecontainersforbothfile sandotherdirectories.
- Whenwecreateanewfilebyopeningitandwriting,thenewfilegoesinthecurrent directory(whereverwewerewhenwerantheprogram).
- Ifwewanttoopenafilesomewhereelse,wehavetospecifythepathtothefile,whic histhenameofthedirectory(orfolder)wherethefileislocated:
- Toaccessanyfilefromthefoldersordirectories,needthefilepath.
- Apathisastringofcharactersusedtouniquelyidentifyalocationinadirectorystru cture.
- **Apath**referstothespecific**locationorroute**throughwhichafileordirectoryca nbeaccessedwithinafilesystem.Example




- Whenworkingwithfilesindirectories,itisagoodideatoletPythondealwithallthe slashesandescapingthem.Theos.pathmoduledoesthisfordifferentoperatingsystems.
- OnWindows,pathsarewrittenusingbackslashes(\)astheseparatorbetweenfol dernames.
- Exampleforpath:C:\Users\asweigart\Documents\project.docx.docx

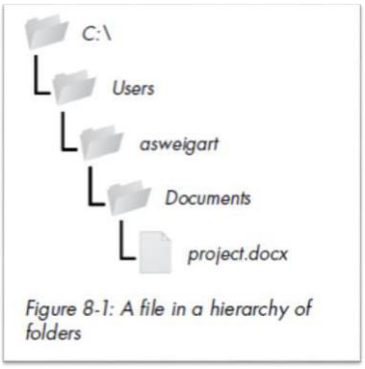



Figure 8-1: A file in a hierarchy of folders

- Ifwewanttoopenafilesomewhereelse,wehavetospecifythepathtothefile,whichisthenameofthedirectory(orfolder)wherethefileislocated:Example

```
>>>wordsfile=open("C:\\usr\\share\\dict//words.txt","r")
>>>wordlist=wordsfile.readlines()
>>>print(wordlist[:6])
    ['\n','A\n',"A's\n",'AOL\n',"AOL's\n",'Aachen\n']
```

**Whataboutfetchingsomethingfromtheweb?**

- Webscrapingistheautomatedprocessofusingsoftwaretoextractlargeamountsofdatafromwebsites.
- WebscrapinginPythoninvolvesprogrammaticallyextractingdatafromwebsitesusingacombinationofPythonlibraries.
- Python'sbuilt-in**urlliblibrary**isafoundationaltoolforwebscraping,primarilyusedtofetchtherawHTMLcontentofawebpage.
- HereisasimpleexamplethatcopiesthecontentsatsomewebURLtoalocalfile.

```
import urllib.request

url = "http://xml.resource.org/public/rfc/txt/rfc793.txt"
destination_filename = "rfc793.txt"

urllib.request.urlretrieve(url, destination_filename)
```

- Theurlretrievefunction— justonecall— couldbeusedtodownloadanykindofcontentfromtheInternet.
- We'llneedtogetafewthingsrightbeforethisworks:

  • The resource we're trying to fetch must exist! Check this using a browser.

  • We'll need permission to write to the destination filename, and the file will be created in the "current directory" - i.e.thesamefolderthatthePythonprogramissavedin.

  • If we are behind a proxy server that requires authentication, (as some studentsare),thismayrequiresomemorespecialhandlingtoworkaroundourproxy.Usealocalresourceforthepurposeofthisdemonstration!

  • We need to make sure that if we use any data from the web, that we checkifthecontentsarestillasweexpectthemtobe.

- Sobeforeyouusedatafromtheinternet,makeyourprogramcheckifthedataiswhatyouwantittobe,beforeexecutinganycodeorshowingittoanyimportantusers!

- Hereisaslightlydifferentexampleusingtherequestsmodule.Thismoduleisnotpartofthestandardlibrarydistributedwithpython,wereaditdirectlyintoastring,andweprintthatstring.Example

```
import requests

url = "http://xml.resource.org/public/rfc/txt/rfc793.txt"
response = requests.get(url)
for line in response:
    print(line)
```

**Module3:QuestionBank**

*1. DefineadictionaryinPython.Howisitdifferentfromalistortuple?**

*2. Explaindictionaryoperationssuchasinsertion,deletion,lookup,andupdatewithexamples.*****

*3. WriteaPythonprogramtocountthefrequencyofeachwordinagivenstringusingadictionary.*****

*4. Whataredictionarymethods?Explainanyfivecommonlyuseddictionarymethodswithexamples.*****

*5. Explainaliasingindictionarieswithasuitableexample.Whycanaliasingleadtounexpectedresults?**

*6. Differentiatebetweenshallowcopyanddeepcopyofadictionary.Illustratedictionarycopyingusingthecopy()method.*****

*7. Writeaprogramtomergetwodictionariesintoone.Ifkeysarecommon,addtheirvalues.*

*8. WhatisNumPy?ListitsadvantagesoverPythonlists.*

*9. ExplaintheconceptofshapeofaNumPyarray.Howcanyouchangetheshapeofanarray?*

*10. WriteaPythonprogramtocreatea2DNumPyarrayanddisplay:****

- *numberofdimensions*
- *shape*
- *datatype*

*11. ExplainarrayslicinginNumPy.HowisslicingdifferentfromslicinginPythonlists?*

*12. WhatismaskinginNumPy?Demonstratehowbooleanindexingworkswithanexample.*

Dr. Suresha

*13. ExplainbroadcastinginNumPy.StatetherulesthatNumPyfollowswhilebroadcastingarrays.*

*14. Writeaprogramtodemonstratebroadcastingbyaddinga1Darraytoa2Darray.*

*15. WhatisdtypeinNumPy?HowcanyouchangethedatatypeofanexistingNumPyarray?*

*16. WhatisafileinPython?Explaindifferentfilemodeswithexamples.*

*17. WriteaPythonprogramtowritedataintoatextfileandthenreaditline-byline.******

*18. Explainthedifferencebetween:****

- *readingafileline-at-a-time*
- *readingthewholefileatonce*
- *Providecodesnippetsforboth.*

*19. Whatarebinaryfiles?Howaretheydifferentfromtextfiles?Writeashortprogramtoreadabinaryfile.*

*20. ExplainhowPythonworkswithdirectories.Writeaprogramto:****

- *listallfilesinadirectory*
- *fetchdatafromawebpageandstoreitinafile*

---


## Acknowledgement:

MysincerethankstotheauthorPeterWentworth,JeffreyElkner,AllenB.DowneyandChris Meyers-,becausetheabovecontentsarepreparedfromtheretextbook**"Howtothinklikeacomputer scientist:learningwithpython3".**GreenTeaPress,Wellesley,Massachusetts,2020





**Preparedby:**

Dr.SureshaVProfessor&

Principal

Dept.ofElectronicsandCommunicationEngineering.Reachmeat:suresha.vee@gmail.com

WhatsApp:+918310992434


---